

SOFTWARE REQUIREMENTS SPECIFICATION

Snake Game

SE Mini Project

Project: Snake Game

Languages/Tech: C/C++ (SFML/Turbo C++)

Version: 1.0

Status: Model Answer / Approved

Team Number: 3

Team Details:

Name	SRN	Contribution
Jaanhavi Ashwin Kher	PES2UG24CS199	External Interface Requirements + RTM
KRISHNA PRAJOTH G	PES2UG24CS237	INTRO AND SYSTEM FEATURES
Krithika.S	PES2UG24CS238	Overall Description and UML use case diagram
Joey Fedrick K	PES2UG24CS206	NFR , Quality attributes & Acceptance tests

Table of Contents

1. Introduction
2. Overall Description
3. External Interface Requirements
4. System Features
5. Non-Functional Requirements
6. Quality Attributes & Acceptance Tests
7. System Models and Diagrams
8. Requirements Traceability Matrix (RTM)

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document defines the requirements for the Snake Game, a single-player, standalone desktop application. It describes the expected functionality, user interaction, system behaviour, and requirements of the game.

The SRS serves as a reference for the development, testing, evaluation, and maintenance of the system. It provides a clear understanding of what the system is expected to do and establishes a common reference for all members of the project team.

The Snake Game allows the Player to control a snake using keyboard input on a grid-based Game Board. The objective is to collect food, increase the snake's length and score, while avoiding collisions with the boundaries of the Game Board and with the snake itself.

The project is developed using C/C++ and is intended to use an SFML or console-based interface, as specified in the master SRS.

1.2 Scope

The Snake Game is a single-player, standalone desktop game in which the Player controls the movement of a snake using the keyboard.

The system shall provide the following major functionality:

- Starting a new game.
- Controlling the snake using directional keyboard input.
- Moving the snake continuously in the selected direction.
- Preventing the snake from making an immediate reverse movement.
- Generating food at a random, valid position on the Game Board.
- Ensuring that newly generated food does not overlap with the snake.
- Detecting food collection by the snake.
- Increasing the snake's length when food is collected.
- Increasing the score by 10 points for each food item collected.
- Detecting collision with the Game Board boundary.
- Detecting collision between the snake's head and its own body.
- Ending the game when a fatal collision occurs.
- Increasing the snake's movement speed as the score increases.
- Pausing and resuming the game using the P key.
- Restarting the game using the R key after Game Over.
- Exiting the application using the Esc key.
- Displaying the current score and relevant Game State.

These decisions are directly aligned with the team's master SRS. The system is limited to a single-player, standalone experience. Network connectivity, multiplayer functionality, database integration, and online leaderboard functionality are outside the scope of this project.

1.3 Intended Audience

- Developers - to understand and implement the required system functionality.
- Testers/QA - to verify that the implemented system satisfies the specified requirements.
- Project Team Members - to maintain a common understanding of the agreed system behaviour.
- Faculty/Evaluators - to evaluate the completeness and correctness of the project requirements and implementation.

1.4 Definitions and Terminology

Term	Definition
Player	The user who controls the snake using keyboard input.
Snake	The controllable game entity that moves across the Game Board and grows when food is collected.
Food	An item placed at a valid random position on the Game Board that can be collected by the snake.
Game Board	The grid-based area in which the snake moves and food is placed.
Score	The numerical value representing the Player's progress. The score increases by 10 points whenever food is collected.
Game Over	The Game State reached when the snake collides with the Game Board boundary or with its own body.
Game State	The current operating state of the game, such as Playing, Paused, or Game Over.
Grid-based Movement	Movement of the snake from one grid position to another rather than unrestricted movement.
Collision	An event in which the snake's head comes into contact with a boundary, its own body, or food.
Pause	A state in which active gameplay is temporarily stopped. The game can be paused or resumed using the P key.
Restart	The process of starting the game again from its initial state after Game Over using the R key.

2. Overall Description

2.1 Product Perspective

The Snake Game is a single-player, standalone desktop application. It is a self-contained product that does not depend on network connectivity, an external database, or any backend/bank-style core service. The Player interacts directly with the system through keyboard input, and the system renders the Game Board, snake, food, score and Game State indicators locally on a single device. The game is implemented in C/C++ and rendered using the SFML graphics library, with a console-based interface available as a fallback presentation layer. No external actor (administrator, database, or server) is involved, consistent with the project's fixed decisions.

2.2 Major Product Functions (Detailed)

The functions below summarise, at a product level, what the system does. Each function is backed by one or more detailed functional requirements (Req IDs SG-F-001 to SG-F-017) specified in Section 4, so that this overview stays consistent with the master requirements list and the traceability matrix in Section 8.

- **Game session control** - Start a new game, pause/resume gameplay, restart after Game Over, and exit the application. (SG-F-001, SG-F-013, SG-F-014, SG-F-015)
- **Snake control & movement** - Accept keyboard directional input, move the snake continuously on the grid, and reject an immediate reverse-direction input. (SG-F-002, SG-F-003, SG-F-004)
- **Food & growth** - Generate food at a valid random position, detect when food is collected, grow the snake, and increase the score. (SG-F-005, SG-F-006, SG-F-007, SG-F-008)
- **Collision & game over** - Detect collision with the Game Board boundary or with the snake's own body, and transition the game to the Game Over state. (SG-F-009, SG-F-010, SG-F-011)
- **Difficulty scaling** - Increase the snake's movement speed as the Player's score increases. (SG-F-012)
- **Feedback & display** - Continuously display the current score and the current Game State to the Player. (SG-F-016, SG-F-017)

2.3 User Roles and Characteristics

Player (sole user role): the end user who operates the keyboard to play the game. No specialised technical knowledge is required. The Player expects short, responsive input latency, clear on-screen feedback of score and Game State, and a simple, predictable control scheme.

In line with the project's fixed decisions, the system supports a single user role only. No administrator, operator, technician, or backend-service actor exists in this project, and the use-case model in Section 7 does not introduce one.

2.4 Operating Environment

- Platform: standalone desktop application (single supported desktop OS/environment).
- Language/runtime: implemented in C/C++.
- Presentation layer: SFML for graphical rendering, with a console-based interface as an alternative build target.
- Connectivity: no network connection required or used.
- Persistence: no database or external storage is used; all Game State is held in memory for the duration of a session.

2.5 Constraints

- Single-player only - no multiplayer mode and no online leaderboard are supported.
- No network communication and no database/back-end integration.
- Keyboard is the only supported input device; controls are fixed as Arrow keys (movement), P (pause/resume), R (restart), and Esc (exit).
- Snake movement is strictly grid-based.
- Score increases by a fixed +10 per food item, and snake speed increases as the score increases; these values are project decisions and may only be changed by first updating the team's master decision sheet.
- An immediate reverse of the snake's current direction of travel is not permitted.
- The system should be implemented using clearly separated, modular components (e.g. Game Controller, Input Handler, Snake Manager, Food Manager, Collision Manager, Score Manager, Game Renderer, Game State Manager, Difficulty Manager) to satisfy the project's maintainability objective (SGNF-005).

2.6 Game States Overview

The system operates as a finite set of Game States, used consistently across the whole SRS and reflected in the use-case model in Section 7:

- **MENU** - Initial state. The Player may start a new game or exit the application.
- **PLAYING** - The snake moves and core game logic (input, growth, scoring, collision detection) is active.
- **PAUSED** - Snake movement and game updates are temporarily suspended; resuming returns to PLAYING.
- **GAME OVER** - Gameplay has ended due to a boundary or self-collision; the Player may restart (returns to PLAYING) or exit.

State transition flow: MENU → PLAYING ⇌ PAUSED; PLAYING → GAME OVER on a fatal collision; from GAME OVER the Player may either Restart (→ PLAYING) or Exit the application.

3. External Interface Requirements

3.1 User Interfaces

The game presents three screens, consistent with the Master SRS Rules UI list. Screen and element names use the terminology defined in Section 1.4 (Player, Snake, Food, Game Board, Score, Game State) so this section reads consistently with Sections 1 and 2.

Main Menu:

- Game title
- Start Game
- Exit

Gameplay Screen:

- Game Board
- Snake
- Food
- Current Score
- Game State indicator (Playing / Paused)
- Pause indicator (shown when paused)

Game Over Screen:

- “Game Over” message
- Final Score
- Restart option
- Exit option

Controls:

Key	Action
↑	Move Up
↓	Move Down
←	Move Left
→	Move Right
P	Pause / Resume
R	Restart after Game Over
Esc	Exit

3.2 Hardware Interfaces

- Standard keyboard - arrow keys for direction, P/R/Esc for pause, restart, and exit.
- Standard display/monitor, rendered via the SFML window (or console output for the console-based build).
- No specialized or custom hardware peripherals are required.

3.3 Software Interfaces

- SFML library (or the console runtime, depending on build target) for rendering and window/input management.
- No external APIs or third-party service integrations - consistent with the “Network: Not required” and “Database: Not required” project decisions.
- No bank-, server-, or cloud-side interfaces of any kind apply to this project.

3.4 Communications

Not applicable. The Snake Game is a standalone, offline, single-player desktop application. It does not transmit or receive data over a network, and does not support multiplayer or an online leaderboard, per the Fixed Project Decisions (Section 1).

4. System Features (Detailed)

This section specifies the detailed functional features of the Snake Game. Each requirement is identified using the agreed SG-F-### requirement ID and includes its priority, acceptance criteria, test case reference, and relevant dependencies.

4.1 Game Initialization and Start

Description: The system shall provide functionality for the Player to start a new game and initialize the required game elements before gameplay begins.

Req ID	Requirement	Priority	Acceptance Criteria / Test Case	Comments / Dependencies
SG-F-001	The system shall allow the Player to start a new game.	High	AC-SG-F-001: Selecting Start begins gameplay with the initial snake, score, and food. Test: TC-START-01	Requires initialization of the Game State, snake, score, and food.

4.2 Snake Movement and Control

Description: The system shall allow the Player to control the direction of the snake using keyboard input. The snake shall move continuously in the selected direction while preventing an immediate reversal of direction.

Req ID	Requirement	Priority	Acceptance Criteria / Test Case	Comments / Dependencies
SG-F-002	The system shall accept keyboard input for controlling snake direction.	High	AC-SG-F-002: Valid directional keys change the snake's movement direction. Test: TC-MOVE-01	Requires keyboard input handling.
SG-F-003	The system shall move the snake continuously in its current direction.	High	AC-SG-F-003: The snake moves at the configured game speed while gameplay is active. Test: TC-MOVE-02	Depends on the current movement direction and game speed.
SG-F-004	The system shall prevent the Player from immediately reversing the snake's direction.	High	AC-SG-F-004: A direct opposite-direction input is ignored while moving. Test: TC-MOVE-03	Prevents invalid immediate reverse movement.

4.3 Food Management

Description: The system shall generate food at valid positions on the Game Board and detect when the snake collects food. Food collection shall increase the snake's length and Player's score.

Req ID	Requirement	Priority	Acceptance Criteria / Test Case	Comments / Dependencies
SG-F-005	The system shall generate food at a valid random position within the Game Board.	High	AC-SG-F-005: Food appears inside the boundary and does not overlap the snake. Test: TC-FOOD-01	Requires valid random-position generation.
SG-F-006	The system shall detect when the snake collects food.	High	AC-SG-F-006: A collision between the snake's head and food is detected. Test: TC-FOOD-02	Depends on collision detection.
SG-F-007	The system shall increase the snake's length after food is collected.	High	AC-SG-F-007: The snake gains one segment after collecting food. Test: TC-GROW-01	Triggered by successful food collection.
SG-F-008	The system shall increase the Player's score when food is collected.	High	AC-SG-F-008: The score increases by 10 after each food collection. Test: TC-SCORE-01	The score increases by 10 for each food item collected.

4.4 Collision Detection and Game Over

Description: The system shall detect collisions involving the snake and determine when gameplay must end. Collision with the Game Board boundary or the snake's own body shall result in Game Over.

Req ID	Requirement	Priority	Acceptance Criteria / Test Case	Comments / Dependencies
SG-F-009	The system shall detect collision between the snake and the Game Board boundary.	High	AC-SG-F-009: Contact with a boundary triggers Game Over. Test: TC-COLL-01	Requires boundary collision detection.
SG-F-010	The system shall detect collision between the snake's head and its body.	High	AC-SG-F-010: Contact with the snake's body triggers Game Over. Test: TC-COLL-02	Requires self-collision detection.

Req ID	Requirement	Priority	Acceptance Criteria / Test Case	Comments / Dependencies
SG-F-011	The system shall transition to Game Over when a fatal collision occurs.	High	AC-SG-F-011: Gameplay stops and Game Over information is displayed. Test: TC-GAMEOVER-01	Triggered by boundary or self-collision.

4.5 Difficulty and Speed

Description: The system shall adjust the snake's movement speed according to the Player's score to progressively increase the difficulty of gameplay.

Req ID	Requirement	Priority	Acceptance Criteria / Test Case	Comments / Dependencies
SG-F-012	The system shall increase snake speed as the Player's score increases.	Medium	AC-SG-F-012: Speed increases according to the configured difficulty rule. Test: TC-SPEED-01	Depends on the current Player score and configured difficulty rule.

4.6 Pause and Resume

Description: The system shall allow the Player to temporarily pause active gameplay and resume it when required.

Req ID	Requirement	Priority	Acceptance Criteria / Test Case	Comments / Dependencies
SG-F-013	The system shall allow the Player to pause and resume gameplay.	Medium	AC-SG-F-013: Pressing P pauses/resumes movement and game updates. Test: TC-PAUSE-01	Uses the P key to toggle pause and resume.

4.7 Restart and Exit

Description: The system shall provide the Player with the ability to restart the game after Game Over and exit the application when required.

Req ID	Requirement	Priority	Acceptance Criteria / Test Case	Comments / Dependencies
SG-F-014	The system shall allow the Player to restart the game after Game Over.	High	AC-SG-F-014: Restart resets the snake, score, speed, food, and Game State. Test: TC-RESTART-01	Restart is available after Game Over and uses the R key.

Req ID	Requirement	Priority	Acceptance Criteria / Test Case	Comments / Dependencies
SG-F-015	The system shall allow the Player to exit the game.	High	AC-SG-F-015: Selecting Exit or pressing Esc closes the application. Test: TC-EXIT-01	Uses the Esc key or available Exit functionality.

4.8 Score and Game State Display

Description: The system shall provide the Player with clear information about the current score and current Game State during gameplay.

Req ID	Requirement	Priority	Acceptance Criteria / Test Case	Comments / Dependencies
SG-F-016	The system shall display the current score during gameplay.	Medium	AC-SG-F-016: The current score is visible and updates after food collection. Test: TC-SCORE-02	Depends on the Score Manager and game interface.
SG-F-017	The system shall display the current Game State to the Player.	Medium	AC-SG-F-017: Start, Playing, Paused, and Game Over states are distinguishable. Test: TC-STATE-01	Depends on the Game State Manager and Game Renderer.

4.9 Summary of System Features

The Snake Game shall provide the Player with a complete gameplay cycle beginning with game initialization and continuing through snake movement, food collection, score updates, collision detection, difficulty progression, pause/resume, restart, and exit.

The functional requirements in this section define the expected behaviour of the system and are identified using the agreed SG-F-#### requirement IDs. Their corresponding test cases follow the team's agreed TC-[FUNCTION]-[NUMBER] naming convention.

The conceptual modules associated with these features include the Game Controller, Input Handler, Snake Manager, Food Manager, Collision Manager, Score Manager, Game Renderer, Game State Manager, and Difficulty Manager. These are conceptual modules used consistently across the SRS and do not necessarily have to correspond to individual C++ classes.

5. Non-Functional Requirements (Detailed)

The Snake Game shall satisfy the following measurable non-functional requirements to ensure acceptable performance, reliability, usability, portability, and maintainability.

Req ID	Requirement	Category	Priority	Acceptance Criteria / Measurement
SGNF-001	The system shall respond to valid Player input within 100 ms under normal conditions.	Performance	High	Response $\leq$ 100 ms. Test: TC-PERF-01
SGNF-002	The system shall maintain a consistent Game State during normal gameplay without unexpected crashes.	Reliability	High	Complete defined gameplay tests without a crash. Test: TC-REL-01
SGNF-003	The system shall provide clear and understandable controls and game-state information.	Usability	Medium	Players can identify controls/status without additional technical instructions. Test: TC-UX-01
SGNF-004	The system shall run on the specified supported desktop environment.	Portability	Medium	Application launches and performs core gameplay functions on supported platforms. Test: TC-PORT-01
SGNF-005	The system shall use modular and logically organized code for major game components.	Maintainability	Medium	Major components can be modified independently where practical. Test: TC-MAINT-01

5.1 Security

Since this is a standalone Snake game, security is scoped realistically rather than following banking-grade requirements.

5.1.1 Security Objectives

ID	Security Objective
SG-SO-001	Protect the integrity of the game's internal state during normal operation.
SG-SO-002	Prevent unsupported inputs or external interfaces from causing invalid game behavior.

5.1.2 Security Requirements

Req ID	Requirement	Type	Priority	Acceptance Criteria / Test Case
SG-SR-001	The system shall validate Player input before processing movement.	Security	Medium	Validated before movement processing. Test: TC-SEC-01

Req ID	Requirement	Type	Priori ty	Acceptance Criteria / Test Case
SG-SR-002	The system shall ignore invalid or unrecognized input commands.	Security	Mediu m	Invalid/unrecognized commands are ignored. Test: TC-SEC-02
SG-SR-003	The system shall prevent unsupported input from directly modifying Game State.	Security	Mediu m	Unsupported input cannot directly modify Game State. Test: TC-SEC-03
SG-SR-004	The system shall not transmit Player or gameplay data over a network unless explicitly required.	Security	Mediu m	No gameplay data is transmitted over a network. Test: TC-SEC-04
SG-SR-005	The system shall initialize game variables to defined values before gameplay begins.	Security	Mediu m	Game variables have defined initial values before gameplay. Test: TC-SEC-05

6. Quality Attributes & Acceptance Tests

The Snake Game shall be evaluated against the following quality attributes and acceptance tests.

Quality Attribute	Acceptance Criteria	Test Case
Performance	Valid Player input response shall be ≤ 100 ms under normal conditions.	TC-PERF-01
Reliability	The game shall maintain a consistent Game State and complete defined gameplay tests without an unexpected crash.	TC-REL-01
Usability	Player shall be able to identify controls and game status without additional technical instructions.	TC-UX-01
Portability	The application shall launch and perform core gameplay functions on the specified supported desktop environment.	TC-PORT-01
Maintainability	Major game components shall be modular and logically organized.	TC-MAINT-01
Security	All five security requirements shall be verified using their corresponding security tests.	TC-SEC-01 to TC-SEC-05

6.1 Acceptance / Exit Criteria

- All high-priority functional requirements have been implemented and verified.
- No critical NFR failures remain.
- The Requirements Traceability Matrix links requirements to their corresponding test cases.
- All defined acceptance tests have passed.

6.2 Test Case Naming Convention

Test cases shall follow the naming convention TC-[FUNCTION]-[NUMBER]. Test case IDs should clearly identify the function and sequence number.

Examples: TC-START-01, TC-MOVE-01, TC-FOOD-01, TC-COLL-01, TC-SCORE-01, TC-PAUSE-01, TC-RESTART-01, TC-PERF-01, TC-SEC-01.

7. System Models and Diagrams

7.1 UML Use-Case Diagram

The use-case model has a single actor, Player, consistent with Section 2.3 and the project's fixed decisions. No Administrator, Database, or Server actor is introduced, as the system is a standalone, offline, single-player application. Two complementary diagrams are provided: a main, Player-facing diagram (Figure 7.1) covering the primary use cases a Player can invoke, and a system-behaviour diagram (Figure 7.2) showing how those primary use cases include or extend the internal use cases that realise the functional requirements in Section 4.

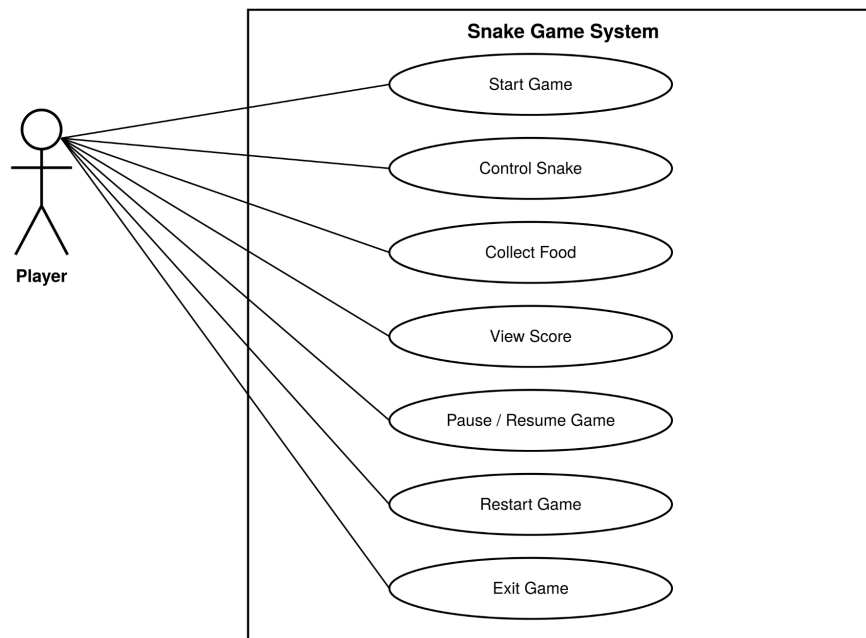


Figure 7.1 Main Use-Case Diagram (Player-facing)

The Player can start a new game, control the snake, collect food, view the current score, view the current game state, pause/resume, restart after Game Over, and exit the application - the eight top-level use cases defined for the Snake Game system.

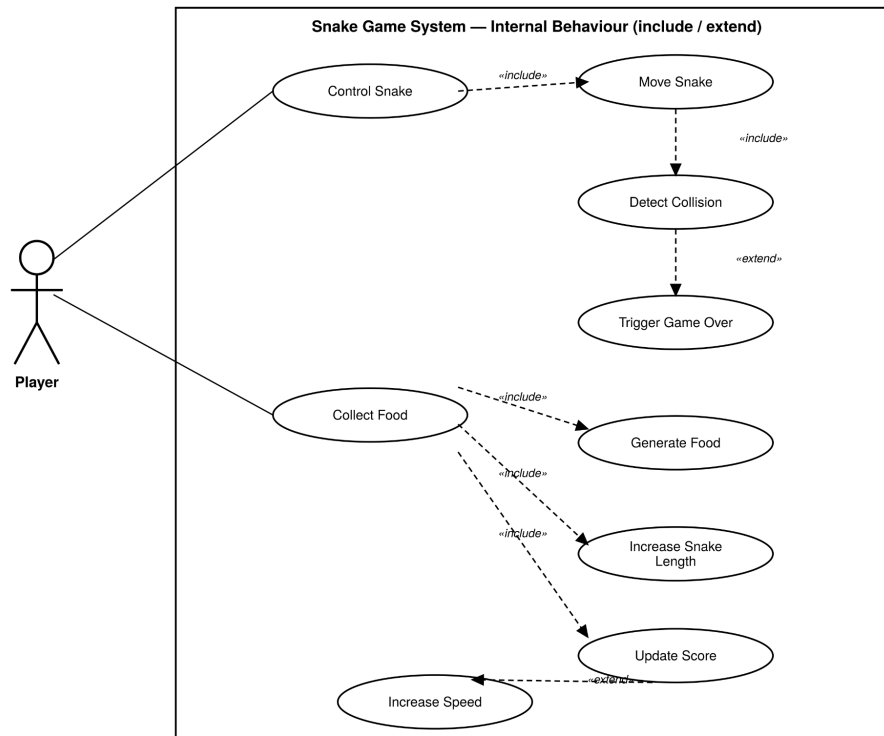


Figure 7.2 System-Behaviour Use-Case Diagram (include / extend relationships)

Figure 7.2 elaborates the internal system behaviour that Section 4's functional requirements describe: Control Snake «includes» Move Snake, which in turn «includes» Detect Collision; a detected collision «extends» to Trigger Game Over. Collect Food «includes» Generate Food, Increase Snake Length and Update Score; an updated score «extends» to Increase Speed. These relationships mirror the master functional requirement list and keep Section 7 traceable to Section 4.

Use Case to Requirement Traceability

The table below maps each use case shown in Figures 7.1 and 7.2 to its related requirement ID(s) from the master Functional Requirements list (Section 4), to support the Requirements Traceability Matrix in Section 8.

Use Case	Type	Related Req ID(s)
Start Game	Player-facing	SG-F-001
Control Snake	Player-facing	SG-F-002, SG-F-004
Collect Food	Player-facing	SG-F-006
View Score	Player-facing	SG-F-016
Pause / Resume Game	Player-facing	SG-F-013
Restart Game	Player-facing	SG-F-014
Exit Game	Player-facing	SG-F-015
View Game State	Player-facing	SG-F-017
Move Snake	Internal (include)	SG-F-003
Generate Food	Internal (include)	SG-F-005
Increase Snake Length	Internal (include)	SG-F-007

Use Case	Type	Related Req ID(s)
Update Score	Internal (include)	SG-F-008
Detect Collision	Internal (include)	SG-F-009, SG-F-010
Trigger Game Over	Internal (extend)	SG-F-011
Increase Speed	Internal (extend)	SG-F-012

8. Requirements Traceability Matrix (RTM)

Every functional, non-functional, and security requirement is traced below to its SRS section, owning module, and test case. Status uses N = Not tested, P = Passed, F = Failed.

Req ID	Requirement (short)	SRS Section	Module	Test Case	Status
SG-F-001	Start new game	4.1	Game Controller	TC-START-01	N
SG-F-002	Accept directional keyboard input	4.2	Input Handler	TC-MOVE-01	N
SG-F-003	Move snake continuously	4.2	Snake Manager	TC-MOVE-02	N
SG-F-004	Prevent immediate reverse direction	4.2	Input Handler	TC-MOVE-03	N
SG-F-005	Generate food at valid random position	4.3	Food Manager	TC-FOOD-01	N
SG-F-006	Detect food collection	4.3	Collision Manager	TC-FOOD-02	N
SG-F-007	Increase snake length after food collected	4.3	Snake Manager	TC-GROW-01	N
SG-F-008	Increase score after food collected	4.3	Score Manager	TC-SCORE-01	N
SG-F-009	Detect Game Board boundary collision	4.4	Collision Manager	TC-COLL-01	N
SG-F-010	Detect self-collision	4.4	Collision Manager	TC-COLL-02	N
SG-F-011	Transition to Game Over on fatal collision	4.4	Game State Manager	TC-GAME OVER-01	N
SG-F-012	Increase speed as score increases	4.5	Difficulty Manager	TC-SPEED-01	N
SG-F-013	Pause and resume gameplay	4.6	Game State Manager	TC-PAUSE-01	N
SG-F-014	Restart game after Game Over	4.7	Game Controller	TC-RESTART-01	N
SG-F-015	Exit the game	4.7	Game Controller	TC-EXIT-01	N
SG-F-016	Display current score	4.8	Score Manager / Game Renderer	TC-SCORE-02	N
SG-F-017	Display current Game State	4.8	Game State Manager / Game Renderer	TC-STATE-01	N
SGNF-001	Input response $\leq$ 100 ms	5	Input Handler / Game Renderer	TC-PERF-01	N
SGNF-002	Consistent Game State, no crashes	5	Entire System	TC-REL-01	N
SGNF-003	Clear, understandable controls/status	5	Game Renderer	TC-UX-01	N

Req ID	Requirement (short)	SRS Section	Module	Test Case	Status
SGNF-004	Runs on supported desktop environment	5	Entire System	TC-PORT-01	N
SGNF-005	Modular, maintainable code	5	All Modules / Codebase	TC-MAINT-01	N
SG-SR-001	Validate Player input before movement	5.1.2	Input Handler	TC-SEC-01	N
SG-SR-002	Ignore invalid/unrecognized input	5.1.2	Input Handler	TC-SEC-02	N
SG-SR-003	Prevent unsupported input from altering state	5.1.2	Game Controller / Input Handler	TC-SEC-03	N
SG-SR-004	No network transmission of gameplay data	5.1.2	Entire System	TC-SEC-04	N
SG-SR-005	Initialize game variables before gameplay	5.1.2	Game Controller	TC-SEC-05	N

All 17 functional requirements (SG-F-001–017), all 5 non-functional requirements (SGNF-001–005), and all 5 security requirements (SG-SR-001–005) are represented above.